
JAVA YAZILIM UZMANI

MÜLAKAT ÖZETİ

Kapsamlı Soru-Cevap Çalışma Notu

Hazırlanan: Batur için

Hedef: Akbank Java Yazılım Uzmanı

Bu döküman; Java temelleri, Java 8/17/21 yenilikleri, Spring Boot, JPA/Hibernate, SQL, NoSQL, Microservice mimarisi, Concurrency ve Design Pattern konularındaki tipik mülakat sorularını ve mülakatçıyı tatmin edecek düzeyde cevaplarını içerir.

İçindekiler

- 1. Kendini Tanıtma Şablonu** — Mülakat açılışı için hazır metin
- 2. Java Temelleri** — JVM, equals/hashCode, String, Exception, GC
- 3. Java 8 / 17 / 21 Yenilikleri** — Stream, Lambda, Records, Virtual Threads
- 4. Collections Derinlemesine** — HashMap iç yapısı, ConcurrentHashMap, Big-O
- 5. OOP, SOLID ve Design Patterns** — Singleton, Factory, Strategy, Observer, Proxy
- 6. Multithreading & Thread Safety** — Lock'lar, volatile, atomic, deadlock
- 7. Spring & Spring Boot** — DI, Bean lifecycle, @Transactional, AOP
- 8. REST API & JWT** — HTTP metotları, status code, güvenlik
- 9. JPA / Hibernate / ORM** — Entity lifecycle, N+1, lock stratejileri
- 10. SQL & Database** — JOIN, Index, ACID, isolation, anomaliler
- 11. NoSQL & MongoDB** — CAP teoremi, ne zaman tercih edilir
- 12. Monolith vs Microservice** — Avantaj/dezavantaj, ne zaman hangisi
- 13. Microservice Pattern'leri** — API Gateway, Eureka, Circuit Breaker, Saga
- 14. Caching** — Local vs distributed, Redis, @Cacheable
- 15. Test (Kısa)** — Unit test, Mockito temelleri
- 16. Clean Code & Refactoring** — Code smell, isimlendirme, refactor
- 17. Behavioral / Soft Skill Soruları** — Davranışsal sorulara hazır cevaplar
- 18. Sen Ne Sorabilirsin?** — Mülakat sonu, sana puan kazandıracak sorular

1. Kendini Tanıtma Şablonu

Mülakatın başında "Bize kendinizi tanıtır mısınız?" diye sorulur. Aşağıda profiline göre hazırlanmış, 60-90 saniyede söylenecek bir şablon var.

Önerilen Açılış Metni

"Merhaba, ben Batur. Yaklaşık birkaç yıldır Java tabanlı kurumsal yazılım geliştirme alanında çalışıyorum. Aktif olarak **Java 8, 17 ve 21** sürümleriyle proje geliştirdim. Hem **monolith** hem de **microservice** mimarilerinde yer aldım; veri tarafında **PostgreSQL** ve **MongoDB** kullandım.

Son dönemde bir HR ürünü olan ARGUS uygulamasında backend geliştirme yaptım. Burada **Spring, JPA, REST servisler, JWT bazlı authentication** ve **SQL Server** ile çalıştım; izin yönetimi, yetkinlik takibi, toplu veri yükleme gibi modüllerde aktif görev aldım.

Bunun yanında **Saga pattern, SOLID prensipler, design pattern'ler** ve **distributed transaction** konularına ilgi duyuyor, sürekli kendimi geliştiriyorum. Akbank'ta bu deneyimimi daha büyük ölçekli sistemlerde uygulamak ve özellikle finansal sistemlerin tutarlılık ve güvenlik gerekliliklerini öğrenmek istiyorum."

❏ **İpucu:** Tanıtım sırasında bahsettiğin her teknolojiyi (Saga, JWT, Spring vs.) mülakatçı detaylı sorabilir. Bu yüzden ezbere değil, gerçekten bildiğin şeyleri söyle. Bilmediğin teknolojiyi tanıtımına koyma.

Sıkça Sorulan Açılış Sonrası Soruları

S: Şu an çalıştığınız projeyi anlatır mısınız?

C: ARGUS, kurumsal HR portalı. Personel, izin, yetkinlik, eğitim modüllerini içeriyor. Backend Java + JPA, frontend JSP. Ben özellikle izin yönetimi ve REST entegrasyon tarafında çalıştım. Örnek olarak yakın zamanda acil durum kişileri için JWT auth'lu bir REST endpoint yazdım.

S: En zorlandığınız bug neydi, nasıl çözdünüz?

C: İzin onay zamanlarının 00:00:00 olarak kaydedildiği bir bug yaşadım. Sebep, Timestamp döndürmesi gereken bir metodun Date dönmesiydi. java.sql.Timestamp'ın Date'i extend ettiği özelliğinden faydalanarak, metod imzasını bozmadan Timestamp instance döndürerek çözdüm. Bu sayede onlarca yerde kullanılan setter'ları değiştirmeden zaman bilgisi korundu.

S: Neden Akbank?

C: Akbank, Türkiye'de modern teknoloji adaptasyonunda öncü bankalardan biri. Özellikle dijital dönüşüm projeleri, microservice mimarisine geçiş, yapay zeka ve veri odaklı bankacılık konularındaki yatırımlar dikkatimi çekiyor. Kurumsal disiplin + teknik derinliğin birleştiği bir ortamda kariyerimi büyütmek istiyorum.

2. Java Temelleri

Mülakatın ısınma turu. Bu sorulara akıcı cevap vermek seni rahat hissettirir; burada tökezlersen mülakatçı zorlamaya başlar.

JDK, JRE, JVM

S: JDK, JRE, JVM arasındaki fark nedir?

C: JVM (Java Virtual Machine): Bytecode'u (.class) çalıştıran sanal makine. Platform bağımsızlığını sağlar — "Write once, run anywhere".

JRE (Java Runtime Environment): JVM + Java standart kütüphaneleri. Java programını çalıştırmak için yeterlidir.

JDK (Java Development Kit): JRE + compiler (javac) + geliştirme araçları (javadoc, jar, debugger). Java programı geliştirmek için JDK gerekir.

Hiyerarşi: **JDK** $\supset$ **JRE** $\supset$ **JVM**

S: Java neden platform bağımsızdır?

C: .java kaynak kodu, javac ile platform bağımsız **bytecode**'a derlenir. Her işletim sisteminde o platforma özel bir JVM bulunur ve bytecode'u kendi platformuna özgü makine koduna çevirir. Bu nedenle aynı .class dosyası Windows, Linux, Mac'te çalışabilir.

== vs equals() ve hashCode()

S: == ile .equals() arasındaki fark nedir?

C: == her zaman **referans karşılaştırması** yapar. Primitive tiplerde değer karşılaştırması olur (çünkü zaten değer tutarlar).

.equals() Object sınıfından gelir; default davranışı == ile aynıdır ama String, Integer, custom sınıflar bunu **override ederek içerik karşılaştırması** yapar.

```
String a = "batur";
String b = "batur";
String c = new String("batur");

a == b           // true  (String Pool, aynı referans)
a == c           // false (c, heap'te yeni nesne)
a.equals(c)      // true  (içerik aynı)
```

S: String Pool nedir?

C: JVM heap'inin özel bir bölgesi. String literal'ler ("batur" gibi) burada **tek nüsha** tutulur. Aynı içerikli literal yazıldığında JVM yeni nesne yaratmaz, mevcut referansı verir. new String("...") ile yazarsan Pool dışında, heap'te yeni nesne yaratılır. intern() ile Pool'a almak mümkün.

S: equals override edersen niye hashCode'u da override etmelisin?

C: Kontrat: a.equals(b) ise a.hashCode() == b.hashCode() olmak zorunda. Aksi halde HashMap/HashSet bozulur. Aynı içerikli iki nesne farklı bucket'a düşer, map.get(p2) null döner — debug edilmesi cehennem bir bug. Lombok'un @EqualsAndHashCode anotasyonu

ikisini birlikte üretir.

String, StringBuilder, StringBuffer

S: Aralarındaki fark nedir?

Özellik	String	StringBuilder	StringBuffer
Mutable?	Hayır (immutable)	Evet	Evet
Thread-safe?	Evet (immutable)	Hayır	Evet (synchronized)
Performans	Yavaş (yeni nesne)	En hızlı	Sync overhead'i yer
Kullanım	Az değişen metin	Tek thread'de döngüde concat	Multi-thread concat

Pratik: Döngüde String concatenation yapıyorsan StringBuilder kullan. Tek satırlık birleştirme için String'in + operatörü zaten compiler tarafından StringBuilder'a çevriliyor (Java 9+ invokedynamic).

final, finally, finalize

S: Üçü arasındaki fark nedir?

C: final: Bir keyword. Sınıf → extend edilemez. Method → override edilemez. Variable → değer atandıktan sonra değiştirilemez. **NOT:** Referans değişmez ama nesnenin içeriği değişebilir — final List l = new ArrayList(); l.add(...) geçerli.

finally: try-catch bloğu. Exception olsun olmasın **her zaman çalışır**. Kaynak kapatmak, log yazmak için. Java 7+ ile try-with-resources daha temiz alternatif.

finalize: Object sınıfının metodu. Garbage Collector nesneyi temizlemeden önce çağırırdı. **Java 9'dan itibaren deprecated**. Yerine try-with-resources veya Cleaner API kullan.

Exception Handling

S: Checked ve Unchecked Exception farkı?

C: Checked: Compile time'da yakalanması zorunlu. Exception'ın doğrudan alt sınıfları (RuntimeException hariç). Örnek: IOException, SQLException. try-catch veya throws gerektirir.

Unchecked: RuntimeException'ın altındakiler. Compile time'da zorlanmaz. Örnek: NullPointerException, IllegalArgumentException, ArrayIndexOutOfBoundsException. Genelde programlama hatalarını işaret ederler.

S: Exception hiyerarşisi nasıldır?

```
Throwable
├── Error (JVM seviyesi - yakalanmaz: OutOfMemoryError, StackOverflowError)
├── Exception (yakalanır)
│   ├── Checked (IOException, SQLException, ...)
│   └── RuntimeException (Unchecked: NPE, IllegalArgumentException, ...)
```

S: REST API'da exception handling nasıl yapılır?

C: Her endpoint'te try-catch yazmak **kod tekrarına** ve **tutarsız response'lara** yol açar. Modern Spring uygulamasında `@ControllerAdvice` + `@ExceptionHandler` ile **global exception handler** yazılır. Tüm controller'lar için tek noktadan standart bir error response döner.

```
@ControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(EntityNotFoundException.class)
    public ResponseEntity<ErrorDto> handle404(EntityNotFoundException ex) {
        return ResponseEntity.status(404).body(new ErrorDto("NOT_FOUND",
ex.getMessage()));
    }
}
```

Garbage Collector

S: Java'da Garbage Collector nasıl çalışır?

C: JVM heap'i iki ana bölgeye ayrılır: **Young Generation** (Eden + 2 Survivor) ve **Old Generation**. Yeni objeler Eden'a düşer. Eden dolunca **Minor GC** tetiklenir; yaşayanlar Survivor'a, belli bir yaşı aştıkları için Old Gen'e **promote** edilir. Old Gen dolunca **Major/Full GC** tetiklenir (daha pahalı). **Stop-The-World:** GC sırasında tüm application thread'leri durur.

S: Hangi GC algoritmaları var?

C: Serial GC: Tek thread, küçük uygulamalar için.

Parallel GC: Çoklu thread, throughput odaklı (Java 8 default).

G1 GC: Heap'i region'lara böler, predictable pause süresi (Java 9+ default).

ZGC, Shenandoah: Ultra-low latency, milisaniyenin altında pause.

S: Memory leak Java'da nasıl olur?

C: GC ortamında bile **kullanılmayan ama referansı tutulan** nesneler bellekte kalır. Klasik örnekler: static collection'a sürekli ekleme yapmak, listener'ları unregister etmemek, ThreadLocal'ı temizlememek, cache'lere TTL koymamak. Profiling için **VisualVM, JProfiler, Eclipse MAT** kullanılır.

Pass by Value vs Reference

S: Java pass-by-value mi, pass-by-reference mi?

C: Java **her zaman pass-by-value'dur**. Ama nesneler için "value" aslında **referansın değeri**'dir. Yani metoda nesne geçtiğinde, referansın *kopyası* geçer. Metot içinde `obj.setX(5)` yaparsan dışarıdaki nesne de etkilenir (aynı objeyi gösteriyor). Ama `obj = new Foo()` yaparsan sadece kopya referans değişir, dışarıdaki etkilenmez.

Autoboxing & Unboxing

S: Autoboxing nedir? Performans etkisi var mı?

C: Primitive'in (int, double) wrapper sınıfa (Integer, Double) otomatik dönüşümü. Tersine **unboxing**. Loop içinde sürekli boxing/unboxing performansı yer, yeni Integer nesneleri yaratılır (Integer cache: -128 ile 127 arası ön belleklenir). Performans kritik kodda `int[]` kullan,

List<Integer> değil.

□ **ipucu:** Mülakatta klasik tuzak: Integer a = 127; Integer b = 127; a == b; → **true**. Ama Integer a = 128; Integer b = 128; a == b; → **false**! Sebep: Integer cache sadece -128..127 arasındır.

3. Java 8 / 17 / 21 Yenilikleri

Modern Java versiyonlarını bilmek, mülakatta seni "güncel kalan geliştirici" olarak konumlandırır.

Java 8 — Fonksiyonel Programlama Devrimi

S: Java 8 ile gelen en önemli yenilikler nelerdir?

C: Lambda expression, Stream API, Optional, Functional interfaces, Method references (::), interface'lerde default ve static metotlar, yeni java.time API, CompletableFuture.

S: Stream API nedir, ne işe yarar?

C: Collection'lar üzerinde **functional** tarzda işlemler yapmayı sağlar. **Lazy evaluation** kullanır; intermediate operation'lar (filter, map) gerçek terminal operation (collect, forEach) çağrılana kadar çalışmaz.

```
List<String> aktifAdlar = personeller.stream()
    .filter(p -> p.isAktif())
    .map(Personel::getAd)
    .sorted()
    .collect(Collectors.toList());
```

S: map() ve flatMap() arasındaki fark nedir?

C: **map()** her elemanı 1-1 başka bir şeye dönüştürür. **flatMap()** her elemanı bir Stream'e çevirir ve sonuçları **tek bir düz stream'e birleştirir**. İç içe yapıları düzleştirmek için kullanılır.

```
// Her personelin birden çok telefonu var, hepsini tek listede istiyoruz
List<String> tumTelefonlar = personeller.stream()
    .flatMap(p -> p.getTelefonlar().stream())
    .collect(Collectors.toList());
```

S: Optional nedir? Yanlış kullanımları nelerdir?

C: NPE riskini açıkça ifade eden wrapper. **Doğru:** metot dönüş tipi. **Yanlış:** field olarak kullanma (Serializable problemi), parametre olarak alma (üç farklı 'null' hali), Collection'a koyma (boş collection zaten yeterli).

```
public Optional<Personel> findByEmail(String email) { ... }

personelService.findByEmail("x@y.com")
    .map(Personel::getAd)
    .orElse("Bulunamadı");
```

S: Functional Interface nedir?

C: Tek abstract method içeren interface. Lambda ile temsil edilebilir. Örnek: Function<T,R>, Predicate<T>, Consumer<T>, Supplier<T>. @FunctionalInterface anotasyonu zorunlu değil ama compiler'a kontrat dayatır.

S: Stream'de findFirst vs findAny farkı?

C: Sequential stream'de aynı sonucu döner. **Paralel stream'de** findAny daha hızlıdır — herhangi bir thread'in bulduğu elemanı döner. findFirst paralel iken bile sıra garantisi verdiği için performans yer.

S: java.time neden geldi, Date'in nesi sorunluydu?

C: Eski java.util.Date ve Calendar: mutable (thread-safe değil), ay 0-indexed, sezgisel olmayan API. Yeni API: **LocalDate**, **LocalDateTime**, **ZonedDateTime**, **Instant**, **Duration**, **Period**. Hepsi immutable ve thread-safe.

Java 17 — LTS, Modern Kurumsal Standart

S: record nedir? Class'tan ne farkı var?

C: Immutable veri taşıyıcısı için kısa syntax. Compiler otomatik olarak constructor, accessor (getter), equals, hashCode, toString üretir. DTO, value object için ideal. Field'lar implicit final, sınıf implicit final, extend edilemez. Interface implement edebilir.

```
public record IzinDTO(Long id, String personelAd, LocalDate baslangic, int gunSayisi) {}

IzinDTO dto = new IzinDTO(1L, "Batur", LocalDate.now(), 5);
dto.personelAd(); // "Batur"
```

S: sealed class/interface ne işe yarar?

C: Bir tipin hangi alt tiplere sahip olabileceğini **kontrollü** belirler. Bu sayede pattern matching switch'i exhaustive (kapsayıcı) olur — yeni alt tip eklenince derleyici tüm switch'leri zorla günceller. Domain modeling için çok güçlü.

```
public sealed interface IzinDurumu
    permits Bekliyor, Onaylandi, Reddedildi {}

public final class Bekliyor implements IzinDurumu {}
public final class Onaylandi implements IzinDurumu {}
public final class Reddedildi implements IzinDurumu {}
```

S: Pattern matching for instanceof nedir?

C: Eskiden: if (obj instanceof Personel) { Personel p = (Personel)obj; ... }. Yeni: if (obj instanceof Personel p) { p.getAd(); } — cast otomatik.

S: Text Blocks ne işe yarar?

C: Çok satırlı String'leri okunaklı yazmak için """" syntax'ı. SQL, JSON, HTML için ideal.

```
String sql = """
    SELECT p.ad, p.soyad
    FROM personel p
    WHERE p.aktif = true
    """;
```

Java 21 — Yeni LTS

S: Virtual Threads (Java 21) nedir?

C: **Project Loom**'un meyvesi. JVM tarafından yönetilen **hafif** thread'ler. Platform thread = 1 OS thread (~1MB stack). Virtual thread = JVM yönetiminde milyonlarca yaratılabilen ucuz

thread. I/O bekleme sırasında otomatik park olur, OS thread'i bloklamaz. **I/O-bound** uygulamalarda thread pool'a gerek kalmadan muazzam concurrency sağlar. CPU-bound'da fayda etmez.

```
// Eski: ExecutorService + sınırlı thread pool
ExecutorService pool = Executors.newFixedThreadPool(200);

// Yeni: Virtual thread, sınırsız ve ucuz
try (var ex = Executors.newVirtualThreadPerTaskExecutor()) {
    IntStream.range(0, 10_000).forEach(i ->
        ex.submit(() -> { httpCall(); return null; })
    );
}
```

S: Pattern matching for switch (Java 21) örneği?

```
String aciklama = switch (durum) {
    case Bekliyor b      -> "Onay bekliyor";
    case Onaylandi o     -> "Onaylandı: " + o.tarih();
    case Reddedildi r    -> "Red: " + r.sebep();
};
```

❏ **İpucu:** Mülakatta "hangi Java sürümünü kullanıyorsun?" sorusuna 21 diyorsan; virtual threads, records, sealed ve pattern matching başlıklarını 1'er cümleyle anlatmaya hazır ol.

4. Collections Derinlemesine

ArrayList vs LinkedList

S: Aralarındaki fark? Hangisini ne zaman kullanırsın?

C: ArrayList: İçeride Object[] dizi tutar. Kapasite dolunca kendini 1.5x büyütür (resize işlemi pahalı). Random access çok hızlı.

LinkedList: Doubly-linked list. Her elemanın prev/next pointer'ları var. Eklemeler hızlı, indexli erişim yavaş.

İşlem	ArrayList	LinkedList
get(index)	O(1)	O(n) — baştan veya sondan gez
add (sona)	O(1) amortized	O(1)
add(index, x) ortaya	O(n)	O(n) gez + O(1) ekle
remove(index)	O(n)	O(n)
Bellek	Verimli (sadece dizi)	Her node için extra pointer

Pratik tercih: Çoğu zaman ArrayList. LinkedList'i sadece başı/sonu yoğun kullanılan Queue/Deque senaryolarında düşün (ama ArrayDeque genelde daha iyi).

HashMap İç Yapısı

S: HashMap nasıl çalışır?

C: 1) Key'in hashCode()'u hesaplanır.

2) Hash, capacity'ye göre bucket index'ine çevrilir: $\text{index} = (\text{n}-1) \& \text{hash}$ (n = bucket sayısı, 2'nin katı).

3) Bucket'taki entry'ler arasında equals() ile key aranır.

Collision: Aynı bucket'taki entry'ler önce linked list olarak tutulur. Java 8+ ile bucket'taki entry sayısı 8'i geçerse ve toplam kapasite 64+ ise **Red-Black Tree**'ye dönüşür → arama O(n)'den O(log n)'e iner.

S: Load factor nedir?

C: Default 0.75. Doluluk %75'i geçince capacity 2 katına çıkar (**rehashing**). Pahalı işlem; başlangıçta initial capacity vermek performansı artırır: `new HashMap<>(expectedSize / 0.75 + 1)`.

S: HashMap thread-safe mi?

C: Hayır. Concurrent put işlemlerinde resize sırasında infinite loop bile oluşabilir (Java 7'de bilinen bug). Alternatifler:

- **Collections.synchronizedMap(map):** Tüm map'i tek lock'a sarar, düşük performans.
- **ConcurrentHashMap:** Java 8+'da CAS bazlı fine-grained locking, çok daha performanslı.

S: HashMap, TreeMap, LinkedHashMap farkı?

Özellik	HashMap	TreeMap	LinkedHashMap
İç yapı	Hash tablo	Red-Black Tree	Hash + linked list
Sıralama	Sirasız	Key'e göre sıralı	Insertion order
get/put	O(1) ortalama	O(log n)	O(1) ortalama
null key	1 tane	İzin vermez	1 tane
Kullanım	Default seçim	Sıralı veri, range query	LRU cache

S: equals override edersen hashCode'u niye override etmelisin?

C: Kontrat: `a.equals(b)` ise `a.hashCode() == b.hashCode()` olmak zorunda. Aksi halde HashMap'e `put(p1)` yapıp `get(p2)` dediğinde (`p1.equals(p2)`) null döner — farklı bucket'a düşmüştür.

Concurrent Collections

ConcurrentHashMap: Java 8+'da node-level CAS lock'lama. Çoğu read lock'suz. Default tercih.

CopyOnWriteArrayList: Her yazmada iç diziyi kopyalar. Yazma pahalı, okuma çok ucuz ve lock'suz. **Read-heavy** senaryoda ideal (event listener listesi vb).

ConcurrentLinkedQueue: Lock-free, CAS bazlı queue.

S: ConcurrentHashMap vs Collections.synchronizedMap() farkı?

C: `synchronizedMap` tüm map'i tek bir lock'a sarar. `ConcurrentHashMap` node-level fine-grained locking yapar; bir bucket'a yazılırken diğeri serbestçe yazılabilir. Çok daha yüksek throughput.

S: Fail-fast vs fail-safe iterator?

C: Fail-fast: `ArrayList`, `HashMap`'in iterator'u — iterasyon sırasında collection değişirse `ConcurrentModificationException` fırlatır. **Fail-safe:** `CopyOnWriteArrayList`, `ConcurrentHashMap`'in iterator'u — kopya/snapshot üzerinde gezdiği için exception fırlatmaz ama yeni değişiklikleri görmez.

5. OOP, SOLID ve Design Patterns

OOP'nin 4 Temel Prensipleri

S: OOP'nin 4 prensibini açıkla mısın?

C: Encapsulation: Veriyi private yapıp dışarıya getter/setter ile kontrollü erişim. Implementation detaylarını gizler.

Inheritance: Bir sınıfın başka bir sınıfın özelliklerini miras alması. "is-a" ilişkisi.

Polymorphism: Aynı isimli metodun farklı davranışlar sergilemesi. Override (runtime, dynamic dispatch) ve overload (compile time) ile sağlanır.

Abstraction: İşin nasıl yapıldığını gizleyip ne yapıldığını öne çıkarmak. Abstract class veya interface ile.

S: Encapsulation ile Abstraction farkı?

C: Encapsulation implementation seviyesinde "veriyi gizleme" (private + getter).

Abstraction tasarım seviyesinde "karmaşıklığı gizleme" (interface/abstract class). Encapsulation 'nasıl saklandığını', abstraction 'nasıl çalıştığını' gizler.

S: Abstract class ile interface farkı?

Özellik	Abstract Class	Interface
Method gövdesi	Var	Java 8+: default & static, ayrıca abstract
Field	Her tip (state tutar)	Sadece public static final
Constructor	Var	Yok
Çoklu kalıtım	Hayır	Evet
İlişki	is-a	can-do
Java 9+	—	private metod eklendi

Tercih: Ortak state + ortak davranış varsa abstract class. Sadece kontrat tanımlıyorsan interface. Java 8'den sonra çoğu durumda interface yeterli.

SOLID Prensipleri

S: SOLID nedir?

C: Robert C. Martin'in 5 prensibi. Sürdürülebilir, esnek kod yazmanın temelidir.

S — Single Responsibility: Bir sınıfın **değişmesi için tek bir nedeni** olmalı. Örnek: PersonelService hem DB işlemi yapıp hem mail gönderiyorsa, iki sorumluluk taşır → PersonelService ve NotificationService diye ayrılmalı.

O — Open/Closed: Sınıflar geliştirmeye açık, değişime kapalı olmalı. Yeni davranış eklerken mevcut kodu değiştirme, sadece yeni kod ekle. Strategy pattern klasik uygulamasıdır.

```
// KÖTÜ: yeni izin tipi gelince if/switch'i değiştir
public double izinOnay(Izin i) {
    if (i.tip.equals("YILLIK")) { ... }
    else if (i.tip.equals("MAZERET")) { ... }
```

```
    else if (i.tip.equals("HASTALIK")) { ... }
}

// İYİ: Strategy + Open/Closed
interface IzinStrategy { boolean onayla(Izin i); }
class YillikIzin implements IzinStrategy { ... }
class MazeretIzin implements IzinStrategy { ... }
// Yeni tip eklemek için yeni sınıf yaz, mevcut koda dokunma
```

L — Liskov Substitution: Alt sınıf, üst sınıfın yerine **davranışı bozmadan** geçebilmelidir. Klasik örnek: Rectangle'dan Square türetirken setWidth(5) çağrısı height'ı da 5 yapmak zorunda kalır → Rectangle bekleyen kod kırılır.

I — Interface Segregation: Bir client kullanmadığı metoda bağımlı olmamalı. Büyük interface'leri parçalara böl. Çalışanın yemek yiyen + maaş alan birleştirilmesi, bir Robot işçisi geldiğinde maaş() metodunu boş override etmek zorunda kalmasına yol açar.

D — Dependency Inversion: Yüksek seviyeli modüller düşük seviyelilere değil, ikisi de **soyutlamalara** bağlı olmalıdır. Detaylar soyutlamalara bağlanır.

```
// KÖTÜ: Doğrudan somut sınıfa bağlı
public class IzinService {
    private MailSender sender = new MailSender();
}

// İYİ: Soyutlamaya bağlı, dışarıdan inject
public class IzinService {
    private final NotificationSender sender;
    public IzinService(NotificationSender s) { this.sender = s; }
}
```

S: DI ile DIP arasındaki ilişki nedir?

C: DIP bir prensip ("soyutlamalara bağlan"). **DI bir teknik** ("bağımlılığı dışarıdan al"). DI, DIP'i pratikte uygulamanın yoludur. DI'sı olmayan DIP zayıftır (soyutlamaya bağlısın ama içeride new yapıyorsan test edilemezsin). DIP'siz DI da yetersizdir (constructor'dan somut sınıf alıyorsan esneklik yok).

Design Patterns

1) Singleton Pattern

S: Singleton nedir, ne zaman kullanılır?

C: Bir sınıftan **JVM süresince yalnızca tek bir nesne** bulunmasını garanti eder. Connection pool, configuration manager, logger gibi global tek-instance gereken yerlerde kullanılır.

S: Thread-safe Singleton nasıl yazılır?

C: Birkaç yol var: (1) Eager initialization — sınıf yüklendiğinde yaratılır. (2) **Double-Checked Locking** — volatile + iki kez null kontrolü. (3) En önerilen: **Enum Singleton** — JVM tarafından thread-safe, serialization-safe, reflection-safe garanti edilir.

```
// Double-Checked Locking
public class Config {
    private static volatile Config instance;
    private Config() {}
}
```

```

    public static Config getInstance() {
        if (instance == null) {
            synchronized (Config.class) {
                if (instance == null) instance = new Config();
            }
        }
        return instance;
    }
}

// Önerilen: Enum Singleton
public enum ConfigEnum {
    INSTANCE;
    public void doSomething() { ... }
}

```

❏ **İpucu:** Spring'de `@Component` / `@Service` / `@Repository` ile işaretli bean'ler default olarak singleton scope'tur. Yani Spring sana bu pattern'i otomatik veriyor. Kendin yazmana genelde gerek yok.

2) Factory Pattern

S: Factory pattern nedir?

C: Nesne yaratma sorumluluğunu client'tan alıp bir **factory sınıfına** devreder. Client somut sınıfı bilmek zorunda değildir, sadece tip belirtir. **Open/Closed** prensibini desteklemek için kullanılır.

```

interface NotificationSender { void gonder(String mesaj); }
class MailSender implements NotificationSender { ... }
class SmsSender implements NotificationSender { ... }

class NotificationFactory {
    public static NotificationSender create(String tip) {
        return switch (tip) {
            case "email" -> new MailSender();
            case "sms" -> new SmsSender();
            default -> throw new IllegalArgumentException();
        };
    }
}

// Kullanım
NotificationSender s = NotificationFactory.create("sms");

```

S: Factory ile Abstract Factory farkı?

C: Factory tek bir ürün döndürür. **Abstract Factory** birbiriyle ilişkili ürün **ailelerini** döner. Örnek: UI tema fabrikası — "Light Theme Factory" hem Button hem Label hem Input için aynı temalı bileşenler üretir.

3) Strategy Pattern

S: Strategy pattern nedir, nerede kullanılır?

C: Bir algoritmayı veya iş kuralını **değiştirilebilir parçalara** ayırır. Her strateji ortak bir interface'i implement eder; context sınıfı runtime'da uygun stratejiyi seçer. **Open/Closed**

prensibinin en yaygın uygulamasıdır. Composition-over-inheritance prensibini destekler.

```
interface OdemeStrategy {
    void ode(BigDecimal tutar);
}
class KrediKartiOdeme implements OdemeStrategy { ... }
class EFTOdeme implements OdemeStrategy { ... }
class CryptoOdeme implements OdemeStrategy { ... }

class OdemeService {
    private final OdemeStrategy strategy;
    public OdemeService(OdemeStrategy s) { this.strategy = s; }
    public void odemeYap(BigDecimal t) { strategy.ode(t); }
}
```

4) Observer Pattern

S: Observer pattern nedir?

C: Bir **Subject** (Publisher) nesnesinin state'i değiştiğinde, kayıtlı tüm **Observer**'lar (Subscriber) otomatik olarak haberdar edilir. Publisher-Subscriber mantığı. UI event listener'lar, Spring'in ApplicationEventPublisher'ı bu pattern'in örnekleridir.

```
// Spring tarzı Observer:
@Component
public class IzinService {
    @Autowired ApplicationEventPublisher publisher;
    public void onayla(Izin izin) {
        // ... iş mantığı
        publisher.publishEvent(new IzinOnaylandiEvent(izin));
    }
}

@Component
public class Maillistener {
    @EventListener
    public void onIzinOnaylandi(IzinOnaylandiEvent e) {
        // mail gönder
    }
}
```

5) Proxy Pattern

S: Proxy pattern nedir, nerelerde kullanılır?

C: Bir nesneye **vekil (proxy)** bir nesne üzerinden erişim sağlar. Proxy, gerçek nesneye delege etmeden önce/sonra ek davranış ekleyebilir. Tipik kullanımlar:

- **Virtual Proxy:** Lazy loading (Hibernate'in lazy initialization'ı)
- **Security Proxy:** Yetki kontrolü
- **Logging Proxy:** Metot çağrılarını loglamak
- **Remote Proxy:** Network üzerinden erişim (RMI, gRPC)

Spring AOP ve **@Transactional** aslında dinamik proxy üzerinden çalışır.

```
interface PersonelService {
    Personel getById(Long id);
}
class PersonelServiceImpl implements PersonelService { ... }
```



```
class LoggingProxy implements PersonelService {
    private final PersonelService target;
    public LoggingProxy(PersonelService t) { this.target = t; }
    public Personel getById(Long id) {
        log.info("Call: getById({})", id);
        long t0 = System.currentTimeMillis();
        Personel result = target.getById(id);
        log.info("Took {} ms", System.currentTimeMillis() - t0);
        return result;
    }
}
```

S: Spring'de @Transactional nasıl çalışır?

C: Spring, @Transactional bulunduğu bean'leri runtime'da bir **dinamik proxy**'ye sarar. Sen orderService.placeOrder() çağırırsan aslında proxy'yi çağırırsın; proxy önce transaction başlatır, sonra gerçek metoda delege eder, sonunda commit/rollback eder. **Bu yüzden @Transactional private metotta çalışmaz** (proxy tarafından sarılamaz). Aynı sınıf içindeki self-invocation da çalışmaz (proxy bypass edilir).

6. Multithreading & Thread Safety

Bankacılık alanında en kritik konulardan biri. Doğru cevap verebilirsen senior düzey değerlendirme alırsın.

Thread Temelleri

S: Thread ile Runnable arasındaki fark nedir?

C: Thread bir *sınıftır*. **Runnable** bir *functional interfacedir*, sadece `run()` metodu vardır. **Runnable tercih edilir** çünkü Java single inheritance destekler — Thread extend edersen başka sınıfı miras alamazsın, ama Runnable implement edersen serbestsin. Ayrıca Runnable, ExecutorService gibi modern API'lara doğrudan verilebilir.

S: Callable, Runnable'dan ne farklıdır?

C: Callable<V> bir değer döndürür ve checked exception fırlatabilir. V `call()` throws Exception imzasındadır. **Runnable** ise void döner, exception fırlatamaz. `ExecutorService.submit(callable)` bir `Future<V>` döner.

S: Thread yaşam döngüsü (state) nelerdir?

C: NEW → `start()` çağrılmadı.
RUNNABLE → çalışmaya hazır (running de buraya dahil).
BLOCKED → monitor lock bekliyor.
WAITING → başka thread'i süresiz bekliyor (`wait()`, `join()`).
TIMED_WAITING → süreli bekleme (`sleep(ms)`, `wait(ms)`).
TERMINATED → bitti.

Senkronizasyon Araçları

S: synchronized keyword nasıl çalışır?

C: JVM seviyesinde **intrinsic lock** (monitor) alır. Bir thread bloğa girerken lock'ı alır, çıkarken bırakır. Diğer thread'ler bekler. Method-level (`synchronized void f()`) veya block-level (`synchronized(obj) { ... }`) kullanılır. Static method'da lock Class nesnesi üzerindedir.

S: synchronized ile ReentrantLock farkı?

Özellik	synchronized	ReentrantLock
Lock alma	Otomatik (girer-çıkır)	Manuel (<code>lock()/unlock()</code>)
Timeout	Yok	<code>tryLock(t, unit)</code>
Fair lock	Yok	<code>new ReentrantLock(true)</code>
Interrupt	Yok	<code>lockInterruptibly()</code>
Condition	<code>wait/notify (Object)</code>	<code>newCondition()</code> — birden fazla
Sözdizimi	Basit	<code>try-finally</code> şart

```
ReentrantLock lock = new ReentrantLock();
lock.lock();
try {
    // critical section
} finally {
    lock.unlock(); // ASLA unutma!
}
```

S: volatile keyword ne işe yarar? synchronized'dan farkı?

C: **volatile** sadece **visibility** garantisi verir — bir thread'in değişikliği diğer thread'lere anında görünür olur (CPU cache yerine main memory'den okuma). **Atomicity garantisi YOKTUR.** volatile int counter; counter++; thread-safe DEĞİL çünkü counter++ aslında read-modify-write'tir.

synchronized hem visibility hem atomicity sağlar ama lock overhead'i vardır.

❑ **İpucu:** volatile'in doğru kullanımı: tek thread yazar, çoklu thread okur senaryosunda flag — örn. private volatile boolean shutdownRequested;

S: Atomic sınıflar nedir?

C: Lock kullanmadan thread-safe operasyon sağlarlar. CPU'nun **CAS (Compare-And-Swap)** instruction'ını kullanır. AtomicInteger, AtomicLong, AtomicReference, AtomicBoolean. Yüksek contention için LongAdder daha iyi (striping yapar).

```
AtomicInteger counter = new AtomicInteger(0);
counter.incrementAndGet(); // atomic, lock-free
counter.compareAndSet(5, 10); // 5 ise 10 yap, atomic
```

S: ReadWriteLock ne zaman kullanılır?

C: Çok-okuma az-yazma senaryolarında. Aynı anda birden çok okuyucu olabilir, yazıcı tekildir ve tüm okuyucuları bloklar. Cache senaryoları için ideal.

```
ReadWriteLock rw = new ReentrantReadWriteLock();

// Çoklu okuma:
rw.readLock().lock();
try { return cache.get(key); }
finally { rw.readLock().unlock(); }

// Tek yazıcı:
rw.writeLock().lock();
try { cache.put(key, val); }
finally { rw.writeLock().unlock(); }
```

Thread Pool ve Executor

S: ExecutorService ne işe yarar?

C: Thread oluşturmak pahalı bir OS operasyonudur. ExecutorService thread'leri **havuzda** tutar ve yeniden kullanır. Çeşitleri:

- **FixedThreadPool(n):** Sabit n thread.
- **CachedThreadPool:** İhtiyaca göre büyüyen, 60 sn idle thread'i öldüren havuz.
- **SingleThreadExecutor:** Tek thread, görevler sıralı.
- **ScheduledThreadPool:** Zamanlı/periyodik görevler için.

S: submit() ile execute() farkı?

C: execute() **Runnable** alır ve void döner. Exception fırlarsa default uncaught exception handler'a gider.

submit() Runnable/Callable alır, Future döner. Exception fırlarsa Future.get() çağrıldığında ExecutionException içinde gelir.

S: CompletableFuture ne işe yarar?

C: Java 8 ile gelen **async programming** aracı. Eski Future blocking'di (get() ile beklemek zorundaydın). CompletableFuture non-blocking, **callback chain** destekler — bir async işin sonucunu başka async işlere zincirleyebilirsin. Mikroservis çağrılarını paralel yapıp birleştirmek için idealdir.

```
CompletableFuture<User> userFut = CompletableFuture.supplyAsync(() -> getUser(id));
CompletableFuture<Order> orderFut = CompletableFuture.supplyAsync(() -> getOrders(id));

userFut.thenCombine(orderFut, (user, order) -> new Dashboard(user, order))
    .thenAccept(dash -> render(dash))
    .exceptionally(ex -> { log.error(ex); return null; });
```

Deadlock ve Diğer Tehlikeler

S: Deadlock nedir, nasıl önlenir?

C: İki (veya daha fazla) thread'in birbirinin lock'ını sonsuza dek beklemesi. **Önleme stratejileri:**

1. **Lock ordering:** Tüm thread'ler lock'ları aynı sırada alsın (örn. her zaman önce lockA, sonra lockB).
2. **Timeout:** tryLock(5, SECONDS) ile süre koyup pes etmek.
3. **Nested lock'tan kaçınmak:** Mümkünse aynı anda iki lock tutmamak.
4. **Yüksek seviye API:** ConcurrentHashMap, AtomicXxx gibi concurrent collection'lar kullanmak.

S: Deadlock'u nasıl tespit edersin?

C: **jstack** <pid> ile thread dump al, "Found one Java-level deadlock" satırını ara. Alternatif: VisualVM, JConsole. ThreadMXBean.findDeadlockedThreads() ile koddan da kontrol edilebilir.

S: Race condition nedir?

C: İki thread aynı paylaşılan değişkene aynı anda erişip yarış yarattığında oluşan tutarsızlık. Örnek: iki thread aynı bakiyeyi okuyup farklı miktarlar düşmesi → biri ezilir. Synchronization veya atomic operasyonlarla çözülür.

Thread Coordination

CountDownLatch: N event tamamlanana kadar bekle (one-shot). Microservice'lerin ayağa kalkmasını bekleyen entegrasyon testleri için.

CyclicBarrier: N thread aynı noktada buluşana kadar bekle (reusable).

Semaphore: Aynı anda en fazla N thread'in geçmesine izin ver. Rate limiting, DB connection pool benzeri senaryolarda.

BlockingQueue: Producer-Consumer için. put() dolu ise bekler, take() boş ise bekler.

□ **İpucu:** Bankacılık mülakatında çok sorulan klasik: "Bir banka transferinde A ve B hesaplarına lock alırken deadlock'u nasıl önlersin?" Cevap: Her zaman hesapların ID'sine göre **küçükten büyüğe** sırayla lock al — bütün thread'ler aynı sırayı izlediği için deadlock olmaz.

7. Spring & Spring Boot

IoC ve Dependency Injection

S: IoC (Inversion of Control) nedir?

C: Klasik kodda nesneler kendi bağımlılıklarını yaratır (new ile). IoC'de bu sorumluluğu bir **container**'a (Spring'in IoC Container'ı) devredersin. Container nesneleri yaratır, yapılandırır, birbirine bağlar. "Hollywood prensibi" — "don't call us, we'll call you".

S: Dependency Injection nedir, türleri nelerdir?

C: Bir sınıfın bağımlılıklarını **dışarıdan** alması. 3 türü var:

1. Constructor Injection (önerilen): Bağımlılıklar constructor'dan geçer, final field kullanabilirsin. Test edilebilirlik en yüksek.

2. Setter Injection: Opsiyonel bağımlılıklar için.

3. Field Injection (önerilmez): @Autowired direkt field'a. final yapamazsın, test zor, circular dependency'yi gizler.

```
// ÖNERİLEN – Constructor Injection
@Service
public class IzinService {
    private final IzinRepository repo;
    private final NotificationSender notif;

    public IzinService(IzinRepository repo, NotificationSender notif) {
        this.repo = repo;
        this.notif = notif;
    }
}
// Spring 4.3+: tek constructor varsa @Autowired anotasyonu bile gereksiz
```

S: @Component, @Service, @Repository, @Controller farkı?

C: Hepsi temelde **@Component**'in özelleştirilmiş versiyonlarıdır, Spring scanning ile bean olarak kayıt edilir. Fark **semantik niyet** ve bazı ek davranışlarda:

- **@Component:** Generic, util/helper sınıfları için.
- **@Service:** Business logic katmanı.
- **@Repository:** Data access. Ek olarak **DB exception'larını DataAccessException'a** çeviren exception translation sunar.
- **@Controller:** MVC controller, view döndürür.
- **@RestController:** @Controller + @ResponseBody — JSON/XML döner.

Bean Lifecycle & Scope

S: Bean lifecycle nedir?

C: 1. **Instantiation** — Spring nesneyi yaratır.

2. **Property injection** — bağımlılıklar set edilir.

3. **BeanPostProcessor.postProcessBeforeInitialization**

4. **@PostConstruct** veya **afterPropertiesSet()** çağrılır.

5. **BeanPostProcessor.postProcessAfterInitialization** — AOP proxy'leri burada wrap edilir.

6. Bean kullanıma hazır.

7. Context kapanırken **@PreDestroy** veya **destroy()** çağrılır.

S: Bean scope'ları nelerdir?

C: • **singleton** (default): Container başına tek instance.

- **prototype**: Her injection'da yeni instance.
- **request**: Her HTTP request için yeni (web).
- **session**: Her HTTP session için yeni (web).
- **application**: ServletContext için tek (web).

S: Singleton bean'in içine prototype inject edersen ne olur?

C: Klasik tuzak: Singleton bir kez yaratıldığı için prototype bağımlılığı da **bir kere** alır — her metod çağrısında yeni prototype oluşmaz. Çözüm: **@Lookup** anotasyonu, **ObjectFactory<T>** veya **Provider<T>** inject etmek.

@Transactional Derinlemesine

S: @Transactional ne işe yarar, nasıl çalışır?

C: Bir metod/sınıfı transactional yapar — ya hep ya hiç. Spring AOP **proxy** üzerinden çalışır. Metod çağrıldığında proxy önce transaction başlatır, sonra gerçek metoda delege eder, exception olmazsa commit eder, RuntimeException olursa rollback eder.

❗ **İpucu: Klasik tuzak:** *@Transactional checked exception'da default rollback YAPMAZ! Sadece RuntimeException ve Error'da rollback eder. Checked exception için @Transactional(rollbackFor = Exception.class) belirtmelisin.*

S: @Transactional propagation türleri nelerdir?

C: Bir transactional metod içinden başka bir transactional metod çağrıldığında ne olacağını belirler.

- **REQUIRED** (default): Aktif tx varsa katılır, yoksa yeni başlatır.
- **REQUIRES_NEW**: Mevcut tx'i askıya alır, yeni bağımsız tx başlatır. Audit log için ideal (ana iş rollback olsa bile log commit olmalı).
- **NESTED**: Savepoint ile iç içe tx.
- **MANDATORY**: Aktif tx olmalı, yoksa hata.
- **SUPPORTS**: Aktif tx varsa katılır, yoksa tx'siz çalışır.
- **NOT_SUPPORTED**: Tx'i askıya alır, tx'siz çalışır.
- **NEVER**: Aktif tx varsa hata.

S: Isolation level'lar nelerdir?

C: Bir tx'in başka tx'lerden ne kadar yalıtık çalıştığını belirler.

- **READ_UNCOMMITTED**: En düşük, dirty read olur.
- **READ_COMMITTED**: Dirty read'i engeller (SQL Server, Postgres default).
- **REPEATABLE_READ**: Aynı sorgu aynı sonucu garantiler.
- **SERIALIZABLE**: Tam izolasyon, en yavaş.

S: @Transactional private metotta neden çalışmaz?

C: Spring proxy bazlıdır. Proxy yalnızca **dışarıdan** gelen public method çağrılarını yakalayabilir. Private metod doğrudan invoke edilir, proxy bypass olur. Aynı sınıfın içinden this.metod() çağrısı da (self-invocation) proxy'yi atlar — @Transactional ETKİSİZ KALIR. Çözüm: metodu başka bean'e taşı veya AspectJ kullan.

Spring Boot Özellikleri

S: Spring Boot, Spring'den ne farklıdır?

C: Auto-configuration: Classpath'teki kütüphaneye göre otomatik bean yapılandırması (örn. H2 varsa DataSource'u otomatik kurar).

Starter'lar: spring-boot-starter-web, -data-jpa gibi paketler — bağımlılık karmaşasını yok eder.

Embedded server: Tomcat/Jetty gömülü, deploy için WAR şart değil; java -jar app.jar yeterli.

Production-ready: Actuator ile health, metrics, info endpoint'leri otomatik.

S: application.yml ile application.properties farkı?

C: Aynı şeyi yaparlar. YAML hierarchik yapı sayesinde daha okunaklı, özellikle profile bazlı (spring.datasource.url gibi) ayarlarda. Property dosyası flat, basit ama hierarchik veride zayıf.

S: Spring Profiles ne işe yarar?

C: Farklı ortamlar (dev, test, prod) için farklı yapılandırma. application-dev.yml, application-prod.yml dosyaları ve spring.profiles.active=prod ile aktive edilir. @Profile("dev") anotasyonu ile bean'leri profil bazlı tanımlayabilirsin.

S: Spring Actuator ne işe yarar?

C: Production monitoring için endpoint'ler: /health (servis sağlığı), /metrics (memory, CPU, HTTP istatistikleri), /info, /env, /loggers. Prometheus + Grafana entegrasyonu için temel.

S: Spring AOP nedir?

C: Aspect-Oriented Programming. Cross-cutting concern'ları (logging, security, transaction) ana iş kodundan ayırmaya yarar. **Aspect** + **Pointcut** (nerede uygulanacak) + **Advice** (ne yapılacak — @Before, @After, @Around) kavramlarıyla çalışır. Spring AOP runtime proxy bazlıdır; AspectJ ise compile-time/load-time weaving yapar.

```
@Aspect
@Component
public class LoggingAspect {
    @Around("execution(* com.batur.service..*(..))")
    public Object log(ProceedingJoinPoint jp) throws Throwable {
        long t0 = System.currentTimeMillis();
        Object result = jp.proceed();
        log.info("{} took {} ms", jp.getSignature(), System.currentTimeMillis() - t0);
        return result;
    }
}
```


8. REST API & JWT

REST Temelleri

S: REST nedir, prensipleri nelerdir?

C: Representational State Transfer. Roy Fielding'in tanımladığı mimari stil. Prensipler:

- 1. Client-Server** ayrımı.
- 2. Stateless:** Server, client state'ini saklamaz; her istek kendi kendine yeterli.
- 3. Cacheable:** Response'lar cache'lenebilir olmalı.
- 4. Uniform Interface:** HTTP metotları + URI standartları.
- 5. Layered System:** Proxy/Gateway ara katmanları olabilir.

S: HTTP metotları ve kullanım alanları?

Metot	Amaç	Idempotent	Body
GET	Veri okuma	Evet	Yok
POST	Yeni kayıt yaratma	Hayır	Var
PUT	Tam güncelleme (replace)	Evet	Var
PATCH	Kısmi güncelleme	Hayır (genelde)	Var
DELETE	Kayıt silme	Evet	Genelde yok
HEAD	GET'in body'siz hali	Evet	Yok
OPTIONS	İzin verilen metotlar	Evet	Yok

S: Idempotency nedir, niye önemli?

C: Aynı isteği N kere atmanın sonucu 1 kere atmakla aynı olmasıdır. Network failure → client retry yaparsa idempotent metot duplicate kayıt yaratmaz. Bu yüzden GET, PUT, DELETE retry-safe; POST ise idempotency-key gerektirir.

S: Önemli HTTP status code'ları?

Code	Anlamı	Tipik kullanım
200	OK	GET, PUT başarılı
201	Created	POST ile kayıt yaratıldı
204	No Content	DELETE başarılı (body yok)
400	Bad Request	Validation hatası, malformed request
401	Unauthorized	Auth gerekiyor / token geçersiz
403	Forbidden	Auth var ama yetki yok
404	Not Found	Kaynak yok
409	Conflict	Concurrent update, optimistic lock

Code	Anlamı	Tipik kullanım
422	Unprocessable Entity	Validation (semantic) hatası
429	Too Many Requests	Rate limit aşıldı
500	Internal Server Error	Sunucu hatası
503	Service Unavailable	Geçici çöküş, scheduled maintenance

- **HttpOnly Cookie:** Daha güvenli, JavaScript erişemez. CSRF için ek önlem gerekir. Her istekte header ile gönderilir: Authorization: Bearer <token>

S: JWT'nin dezavantajları?

- C:**
- **Revoke zorluğu:** Stateless olduğu için bir token'ı süresi dolmadan iptal etmek zor (blacklist tablosu ekleyerek çözülebilir ama stateless avantajını yitirir).
 - **Boyut:** Cookie'den büyük, her istekte gider.
 - **Hassas veri risk:** Payload base64 encoded ama **şifrelenmiş değildir**. Hassas veri koyma.

S: Refresh Token ne işe yarar?

C: Access token kısa ömürlü (örn. 15 dk) tutulur — risk düşer. Süresi dolduğunda client, uzun ömürlü refresh token (örn. 7 gün) ile yeni access token alır. Refresh token DB'de tutulur, gerekirse iptal edilebilir.

S: CORS nedir, nasıl çözülür?

C: Cross-Origin Resource Sharing. Tarayıcı, farklı origin'deki API'ya istek atmayı default engeller. Server Access-Control-Allow-Origin header'ı ile izin verir. Spring'de @CrossOrigin anotasyonu veya WebMvcConfigurer ile global config yapılır.

9. JPA / Hibernate / ORM

ORM ve JPA Temelleri

S: ORM nedir, niye kullanılır?

C: Object-Relational Mapping. İlişkisel DB tablolarını Java sınıflarına eşler. DB satırı → Java nesnesi, kolonlar → field'lar, FK'lar → object referansları. SQL yazmak yerine nesneler üzerinden çalışırsın. Avantajları: kod tekrarı az, DB bağımsızlığı (Postgres → MySQL'e geçiş kolay), güvenlik (SQL injection'a karşı parametrik query), bakım kolaylığı.

S: JPA ile Hibernate farkı nedir?

C: JPA bir spesifikasyondur (Java Persistence API), bir *kontrat* belirler. Tek başına bir şey yapmaz, sadece kuralları söyler. **Hibernate** bu spesifikasyonun en yaygın **implementasyonudur**. Diğer implementasyonlar: EclipseLink, OpenJPA. Pratikte JPA + Hibernate kullanır, JPA anotasyonlarını (@Entity, @Id, @Column) yazarsın, Hibernate çalıştırır.

S: @Entity ne işe yarar?

C: Bir Java sınıfının DB tablosuna eşleştiğini belirtir. Default tablo adı sınıf adıdır, @Table(name = "...") ile değiştirilir. Sınıfın **parametresiz constructor**'ı ve **@Id** ile işaretli bir field'ı olmak zorundadır.

```
@Entity
@Table(name = "personel")
public class Personel {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "ad", nullable = false, length = 100)
    private String ad;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "departman_id")
    private Departman departman;
}
```

Entity Lifecycle

S: Entity'nin 4 yaşam döngüsü durumu nedir?

C: 1. Transient (New): new Personel() ile yarattığın nesne. Persistence Context'te değil, DB'de karşılığı yok.

2. Persistent (Managed): persist() veya save() sonrası. Persistence Context entity'yi izler — üzerinde yaptığın değişiklikler **dirty checking** ile transaction commit'inde otomatik DB'ye yansır.

3. Detached: Persistence Context kapandıktan sonra (session close, tx end). Değişiklikler artık DB'ye yansımaz. merge() ile geri persistent yapılabilir.

4. Removed: remove() ile silinmeye işaretli. Flush'ta DELETE atılır.

S: save() ile saveAndFlush() farkı?

C: save(): Entity'yi Persistence Context'e ekler, SQL hemen DB'ye GİTMEZ — tx commit'te veya flush'ta gider.

saveAndFlush(): Anında **flush** yapar, INSERT/UPDATE hemen DB'ye gönderilir (ama commit değil — hala rollback edilebilir). Aynı tx içinde native SQL ile sorgu atacaksan ve değişikliklerin görünmesi gerekiyorsa saveAndFlush kullan.

S: persist() ile merge() farkı?

C: persist(): Yeni (Transient) entity'yi Persistence Context'e alır, sınıfın id'si null olmalıdır.

merge(): Detached bir entity'yi Persistence Context'e geri alır. Detached entity'nin değerlerini Persistence Context'teki kopyaya kopyalar ve **yeni managed instance döner**. Önemli: merge sonrası elindeki nesne hala detached'tır, dönen nesne managed'dır.

Lazy vs Eager Loading

S: LAZY ve EAGER fetch farkı?

C: EAGER: Entity yüklenirken ilişkili nesneler de hemen yüklenir. **LAZY:** ilişkili nesneler ihtiyaç anında (proxy üzerinden ilk erişimde) yüklenir.

Default değerler: @OneToMany ve @ManyToMany → LAZY. @ManyToOne ve @OneToOne → EAGER.

❏ **İpucu: Pratik tavsiye: HER ZAMAN LAZY tercih et (override ile de olsa). EAGER istemediğin halde verileri çeker, performans baş ağrısı yapar. İhtiyaç olduğunda JOIN FETCH veya @EntityGraph ile selektif eager yap.**

S: LazyInitializationException ne zaman oluşur?

C: Detached bir entity'nin lazy ilişkisine eriştiğinde (Persistence Context kapalı). Tipik: Service katmanı entity'yi çekti, Controller'a döndü, JSON serializer lazy field'a erişti → patlama. Çözümler: **JOIN FETCH** ile baştan çek, **DTO Projection** kullan, Open-Session-In-View (anti-pattern, dikkat) açma.

N+1 Problemi (Mülakatın Klasik Sorusu)

S: N+1 query problem nedir?

C: Bir ana entity listesi çekmek için **1 sorgu** atılır (örn. 100 personel). Sonra her birinin lazy ilişkisine (örn. departman) erişildikçe **N tane ek sorgu** atılır → toplam 1+N. Büyük listelerde DB'yi yorar, response süresini patlatır.

S: N+1 nasıl çözülür?

C: 1. JOIN FETCH (JPQL): Tek sorguda ilişkili veriyi de getirir.

2. @EntityGraph: Repository metoduna deklaratif fetch path tanımı.

3. @BatchSize(size=20): N+1'i N/20+1'e indirir, IN clause ile batch çeker.

4. DTO Projection: Sadece ihtiyacın olan alanları seçen DTO sorgusu.

```
// 1) JOIN FETCH
@Query("SELECT p FROM Personel p JOIN FETCH p.departman")
List<Personel> findAllWithDepartman();

// 2) EntityGraph
@EntityGraph(attributePaths = {"departman", "izinler"})
```

```

List<Personel> findAll();

// 3) BatchSize entity'de
@Entity
@BatchSize(size = 20)
public class Departman { ... }

// 4) DTO Projection
@Query("SELECT new com.batur.PersonelDTO(p.id, p.ad, d.ad) "
      + "FROM Personel p JOIN p.departman d")
List<PersonelDTO> findAllDTO();

```

Lock Stratejileri

S: Optimistic ve Pessimistic locking farkı?

C: Pessimistic Lock ("kötümser"): Veriyi okurken DB seviyesinde kilitle, kimse dokunmasın. SELECT ... FOR UPDATE. Çakışma sık ise tercih. Throughput düşer.

Optimistic Lock ("iyimser"): Lock alma, ama commit anında @Version field'ı ile kontrol et. Başkası araya girmişse OptimisticLockException fırlat. Çakışma nadir ise tercih. Performans yüksek.

```

// Optimistic locking
@Entity
public class IzinBasvuru {
    @Id Long id;
    @Version Long version; // Her update'te otomatik artar
    String durum;
}

// Pessimistic locking
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT i FROM IzinBasvuru i WHERE i.id = :id")
IzinBasvuru findByIdForUpdate(@Param("id") Long id);

```

S: Cascade tipleri ne işe yarar?

C: Parent entity'de yapılan operasyonun ilişkili child'lara da uygulanmasını sağlar.

- **PERSIST:** Parent save edilince child da save edilir.
- **MERGE:** Parent merge olunca child da.
- **REMOVE:** Parent silinince child da silinir.
- **REFRESH:** Parent refresh olunca child da.
- **ALL:** Hepsi.
- **orphanRemoval = true:** Parent'tan koparılan child silinir.

❏ **İpucu: CascadeType.ALL + orphanRemoval'a dikkat!** Yanlış yerde kullanırsan beklenmedik silmeler tetiklenebilir. Klasik tuzak: shared child entity'ye cascade.ALL koymak.

JPA Repository ve Spring Data

S: Spring Data JPA'nın faydaları?

C: JpaRepository'yi extend ederek CRUD metotlarını ÜCRETSİZ alırsın. Method ismi türetme: findByIdAndAktifTrue gibi metot tanımlarsın, Spring SQL'i otomatik üretir. Custom sorgu için @Query kullanılır.

```
public interface PersonelRepository extends JpaRepository<Personel, Long> {
    List<Personel> findByDepartmanIdAndAktifTrue(Long departmanId);
    Optional<Personel> findByEmail(String email);

    @Query("SELECT p FROM Personel p WHERE p.maas > :limit")
    List<Personel> findHighEarners(@Param("limit") BigDecimal limit);
}
```

10. SQL & Database

JOIN Türleri

S: INNER, LEFT, RIGHT, FULL OUTER JOIN farkı?

C: INNER JOIN: İki tablodan da sadece **eşleşen** satırlar gelir.

LEFT JOIN: Sol tablodaki **tüm satırlar** + sağdan eşleşenler. Eşleşme yoksa sağ kolonlar NULL.

RIGHT JOIN: Tam tersi.

FULL OUTER JOIN: İki tablodaki **tüm satırlar**, eşleşme yoksa NULL.

CROSS JOIN: Cartesian product, her satır her satırla eşleşir.

S: Hangi JOIN'i ne zaman kullanırsın?

C: Senaryoya göre: "İzni olmayan personelleri de göster" → **LEFT JOIN** (personel solda, izin sağda). "Sadece izni olan personeller" → **INNER JOIN**. "Her iki taraf eksik olabilir" → **FULL OUTER** (nadiren).

Index

S: Index nedir, niçin kullanılır?

C: DB'nin bir tabloda **hızlı arama** yapmasını sağlayan veri yapısı (genelde B-Tree). Sık WHERE'de geçen, JOIN kolonu olan, ORDER BY'da geçen kolonlara konulur. DB SELECT * FROM personel WHERE email = ? sorgusunda email indexli ise tüm tabloyu taramak yerine doğrudan ilgili satıra gider.

S: Index'in dezavantajları?

C: • **Yazma yavaşlar:** INSERT/UPDATE/DELETE'te index de güncellenir.

• **Disk alanı:** Ekstra storage.

• **Düşük kardinalite kolon:** Cinsiyet, aktif/pasif gibi az çeşitli kolona index atmak **faydasız**; tüm tablonun yarısını taramak hala gerekir.

S: Clustered ve Non-Clustered Index farkı?

C: Clustered Index: Tablonun verisini **fiziksel olarak** bu index sırasına göre saklar. Bu yüzden tabloda **sadece bir tane** olabilir (genelde PK). SQL Server'da PK default clustered'dır.

Non-Clustered Index: Ayrı bir yapıdır, tablo verisine **pointer** tutar. Bir tabloda **birden çok** olabilir.

S: Composite Index nedir?

C: Birden fazla kolonu birlikte indeksleyen yapı: (ad, soyad). **Sıralı çalışır** — kolon sırası önemlidir. (ad, soyad) indeksinde WHERE ad = ? sorgusu hızlı, ama WHERE soyad = ? tek başına index'ten yararlanamaz.

ACID — Banka Mülakatının En Sevdiği Konu

S: ACID nedir?

C: Transaction'ların tutarlılık garantileri:

A — Atomicity: Ya hep ya hiç. Bir tx'in tüm operasyonları başarılı olur ya da hiçbiri.

C — Consistency: Tx sonunda DB geçerli bir state'te olmalı (constraint'ler, FK, business rule bozulmamalı).

I — Isolation: Eş zamanlı tx'ler birbirini bozmasın, sanki sırayla çalışıyormuş gibi görünsün.

D — Durability: Commit edilen tx, sistem çökse bile kaybolmaz.

S: ACID nasıl sağlanır?

C: A: Undo Log (rollback journal). Tx başarısız olursa eski değerler geri yazılır.

C: DB constraint'leri (FK, UNIQUE, CHECK, NOT NULL). Commit anında kontrol edilir.

I: Lock (S/X locks) veya MVCC (Multi-Version Concurrency Control — PostgreSQL/Oracle). Isolation level seçimiyle dengeli ayarlanır.

D: Write-Ahead Log (WAL). Commit'ten önce log dosyasına fsync ile yazılır; crash sonrası WAL replay edilir.

Isolation Level'lar ve Anomaliler

S: Hangi anomaliler vardır?

C: Dirty Read: Commit edilmemiş veriyi okumak. Tx A bakiyeyi güncelledi, rollback yaptı, ama Tx B arada okuyup karar verdi → felaket.

Non-Repeatable Read: Aynı satırı iki kere okuyunca farklı değer görmek. Tx B araya girip commit etti.

Phantom Read: Aynı sorguyu tekrar çalıştırınca farklı satır kümesi gelmesi. Tx B yeni satır ekledi/sildi.

Lost Update: İki tx aynı satırı güncelledi, ikincisi birincinin değişikliğini ezdi.

S: Isolation level'lar hangi anomalileri engeller?

Isolation Level	Dirty Read	Non-Repeatable	Phantom
READ_UNCOMMITTED	Olur	Olur	Olur
READ_COMMITTED	Engellenir	Olur	Olur
REPEATABLE_READ	Engellenir	Engellenir	Olur
SERIALIZABLE	Engellenir	Engellenir	Engellenir

Default değerler: SQL Server, Oracle, PostgreSQL → READ_COMMITTED. MySQL InnoDB → REPEATABLE_READ. Yüksek izolasyon = daha fazla lock = daha düşük throughput.

DDL vs DML — DELETE/TRUNCATE/DROP

S: DELETE, TRUNCATE, DROP farkı?

Komut	Ne yapar?	Rollback?	WHERE?	Hız
DELETE	Satırları siler	Evet (log tutar)	Var	Yavaş
TRUNCATE	Tüm satırları siler, tablo yapısı kalır	DB'ye göre değişir	Yok	Çok hızlı

Komut	Ne yapar?	Rollback?	WHERE?	Hız
DROP	Tabloyu komple siler (yapı + veri)	Genelde hayır (DDL)	Yok	Hızlı

Diğer SQL Konuları

S: Stored Procedure ve Trigger farkı?

C: Stored Procedure: DB'de saklanan, parametre alıp iş yapan kod parçası. Manuel çağırılır (CALL sp_isim(...)). Karmaşık iş mantığı, batch işlemler için.

Trigger: Belirli bir olay (INSERT/UPDATE/DELETE) gerçekleşince **otomatik** tetiklenir. Audit log, otomatik kontrol için kullanılır.

S: View nedir?

C: Saklı bir sorgudur, sanal tablo gibi davranır. Karmaşık JOIN'leri saklayıp tek isimle kullanırsın. **Materialized view** ise sonucu fiziksel olarak saklar, periyodik refresh edilir — pahalı analitik sorgular için performans çözümüdür.

S: Normalizasyon nedir, nereye kadar yapılır?

C: Veri tekrarını azaltmak ve tutarlılığı artırmak için tabloları parçalama. 1NF (atomik değerler) → 2NF (PK'ya tam bağlılık) → 3NF (transitive bağlılık yok) → BCNF. Pratikte 3NF yeterlidir. Performans için bazen **denormalize** de edilir (okuma yoğun OLAP sistemlerde).

S: Primary Key ve Unique Key farkı?

C: PK: Satırı unique olarak tanımlar, **NULL olamaz**, tabloda tek tane olabilir, default clustered index oluşturur.

Unique: Değerin unique olmasını sağlar ama **tek NULL'a** izin verir, tabloda birden çok olabilir.

11. NoSQL & MongoDB

NoSQL Temelleri

S: NoSQL nedir, RDBMS'ten ne farklıdır?

C: İlişkisel olmayan veritabanları. **Schema-less** veya esnek şemalı. Yatay ölçeklenebilir, büyük veri ve yüksek throughput için tasarlanmıştır. 4 ana tip:

1. Document: MongoDB, CouchDB — JSON/BSON döküman.

2. Key-Value: Redis, DynamoDB — anahtar-değer.

3. Column-family: Cassandra, HBase — kolon bazlı.

4. Graph: Neo4j — düğüm ve kenarlar.

S: CAP teoremi nedir?

C: Eric Brewer'ın teoremi: Dağıtık bir sistemde aynı anda en fazla iki tanesi sağlanır:

C — Consistency: Tüm node'lar aynı veriyi aynı anda görür.

A — Availability: Her istek bir cevap alır (eski/yanlış olabilir).

P — Partition Tolerance: Network bölünmesi olsa da sistem çalışmaya devam eder.

Dağıtık sistemde Partition **kaçınılmaz**, yani gerçek seçim CP veya AP arasındadır.

CP: MongoDB (replica set), HBase. Tutarlılık için availability'den ödün verir.

AP: Cassandra, DynamoDB. Eventual consistency.

S: BASE nedir? ACID'den farkı?

C: BASE: Basically Available, Soft state, Eventual consistency. NoSQL felsefesidir. ACID'in tam tutarlılık garantisi yerine "sonunda tutarlı olur" yaklaşımı. Yüksek ölçek için trade-off.

MongoDB Özelinde

S: MongoDB ne zaman tercih edilir?

- Şema esnek/değişken: ürün katalog, kullanıcı profili, log
- Doküman bazlı erişim: tek query'de iç içe veri çekmek (nested JSON)
- Yüksek yazma throughput'u
- Hızlı geliştirme: schema migration sancısı yok
- Coğrafi sorgular: native geo-spatial destek

Tercih ETMEMEN gereken yerler: Karmaşık çok-tablolu JOIN'ler, ACID gereksinim çok yüksek finansal transferler, raporlama/OLAP.

S: MongoDB temel kavramları?

C: Database → **Collection** (RDBMS'in tablosu) → **Document** (BSON, JSON benzeri). Her döküman'ın bir `_id` field'ı vardır. Schema-less — aynı collection'daki dökümanlar farklı field'lara sahip olabilir.

```
// Bir personel dökümanı
{
  "_id": ObjectId("..."),
  "ad": "Batur",
  "departman": { "id": 5, "ad": "BT" },    // nested
  "telefonlar": ["+90...", "+90..."],    // array
  "izinler": [
```

```
{ "tip": "YILLIK", "gun": 5 },
{ "tip": "MAZERET", "gun": 1 }
]
```

S: MongoDB Spring Data ile nasıl kullanılır?

```
@Document(collection = "personel")
public class Personel {
    @Id String id;
    String ad;
    List<String> telefonlar;
}

public interface PersonelRepo extends MongoRepository<Personel, String> {
    List<Personel> findByDepartmanAd(String departman);
}
```

S: MongoDB'de transaction var mı?

C: MongoDB 4.0+ ile **multi-document ACID transaction** desteklenir (replica set/sharded cluster). Ama performans cezası vardır; ana hedef hala single-document atomicity'dir. Çoklu document tx gerekiyorsa MongoDB değil RDBMS düşünmelisin.

S: Embed vs Reference farkı?

C: Embed (nested): İlişkili veriyi aynı döküman içine göm. Tek query, hızlı okuma. Veri birlikte kullanılıyorsa ve büyümeyecekse iyi.

Reference (link): Başka document'ı ID ile referans et. RDBMS FK'ya benzer. Veri büyükse, paylaşılıyorsa veya bağımsız güncelleniyorsa iyi.

Kural: 1:1 ve 1:few → embed. 1:many ve 1:huge → reference.

12. Monolith vs Microservice

S: Monolith nedir?

C: Tüm modüllerin (UI, business logic, DB erişimi, auth) **tek bir kod tabanı + tek bir deployable artifact** olarak çalıştığı mimari. Tek JAR/WAR, tek DB. Klasik kurumsal uygulamaların çoğu monolith olarak başlar.

S: Microservice nedir?

C: Her **business capability**'nin bağımsız servis olarak deploy edildiği, kendi DB'sini kullandığı, aralarında HTTP/gRPC/message broker ile iletişim kurduğu mimari.

Karşılaştırma

Konu	Monolith	Microservice
Deployment	Tek artifact	Bağımsız her servis
Scale	Tüm uygulamayı birlikte	Sadece ihtiyacı olanı
Tech stack	Tek (kilitli)	Polyglot — her servis farklı dil/DB seçebilir
Geliştirme hızı	Başlangıçta hızlı	Sonradan hızlı (takım bağımsızlığı)
Test	Kolay (tek context)	Zor (entegrasyon testleri)
Transaction	DB ACID kolay	Distributed tx — Saga gerek
Debug	Kolay (tek log)	Zor (distributed tracing gerek)
DevOps	Düşük overhead	Kubernetes, observability şart
Network	In-process call	Network latency var
Fault isolation	Tek modül crash → tüm sistem	Tek servis çöker, diğerleri ayakta

Ne Zaman Hangisi?

Monolith tercih et:

- Küçük takım (< 10 dev)
- MVP, startup, henüz product-market fit yok
- Basit domain
- DevOps olgunluğu düşük
- ACID transaction'lar kritik

Microservice tercih et:

- Büyük takım, çok ekipli organizasyon
- Karmaşık domain, açık bounded context'ler
- Yüksek trafik, modüllerin farklı scale ihtiyaçları
- Sık ve bağımsız deploy gereksinimi
- DevOps + observability olgunluğu mevcut

□ **İpucu:** Martin Fowler'ın "Monolith First" felsefesi: Önce monolith yaz, modülleri öğren, doğru sınırları keşfet, SONRA microservice'lere ayır. Erken microservice felaket olabilir.

S: Modular Monolith nedir?

C: Tek deployable ama içinde modüller arası **net sınırlar** olan mimari. Her modül kendi paketi, kendi public API'sı, ideally kendi DB schema'sı. Microservice avantajlarının çoğunu (sınırlar, takım bağımsızlığı) sunar, dezavantajlarını ortadan kaldırır. Orta ölçekli projeler için altın orta.

S: 12-Factor App nedir?

C: Cloud-native uygulama prensipleri. En önemlileri: Config environment variable'da, Stateless processes, Logs stdout'a, Build-Release-Run ayrımı, Disposable processes (fast startup, graceful shutdown), Dev/prod parity.

13. Microservice Pattern'leri

API Gateway

S: API Gateway nedir, ne işe yarar?

C: Tüm client isteklerinin **tek giriş noktası**. Client'lar bireysel microservice'leri bilmek zorunda değil, sadece Gateway'i bilir. Sorumlulukları:

- **Routing:** İsteği doğru servise yönlendirme
- **Authentication/Authorization:** JWT validation tek noktada
- **Rate limiting:** DDoS koruması
- **Logging & Monitoring:** Tüm trafiği görme
- **Response aggregation:** Birden çok servisten veri toplayıp birleştirme
- **SSL termination:** HTTPS Gateway'de kesilir, içeride HTTP olabilir

Örnek: Spring Cloud Gateway, Kong, AWS API Gateway, Netflix Zuul.

Service Discovery

S: Service Discovery niye gerekli?

C: Container/cloud ortamında servis IP'leri dinamik olarak değişir. "OrderService nerede?" sorusunun cevabı URL hard-code edilemez. Service Discovery, servislerin kendilerini bir **registry**'ye kaydetmesi ve başka servislerin isimle (ID ile değil) erişmesi sistemidir. Auto-scaling, blue-green deployment, failover için zorunlu.

Örnekler: **Netflix Eureka**, **Consul**, **Etcd**, Kubernetes'in kendi DNS'i.

```
// Servis kendini Eureka'ya kaydeder
@EnableEurekaClient
@SpringBootApplication
public class OrderServiceApp { ... }

// Başka servis isimle çağırır (URL değil)
@FeignClient(name = "payment-service")
public interface PaymentClient {
    @PostMapping("/charge")
    PaymentResult charge(@RequestBody ChargeRequest r);
}
```

Load Balancer

S: Load Balancer türleri?

C: Server-side: Nginx, AWS ELB, HAProxy. Client tek IP'ye gider, LB arkadaki instance'lara dağıtır.

Client-side: Spring Cloud LoadBalancer (eski Ribbon). Client, registry'den instance listesini alır, kendisi seçer. Ekstra hop yok, daha hızlı.

Algoritmalar: Round-robin, weighted, least connections, random, IP hash.

Circuit Breaker

S: Circuit Breaker pattern nedir?

C: Bir servis çöktüğünde **cascading failure**'i önler. Elektrikteki sigorta mantığı:

- **Closed (sağlıklı):** İstekler geçer, hatalar sayılır.
- **Open:** Hata oranı eşiği aştı → istekler hemen reddedilir, fallback döner. Bekleyen kullanıcılar bağlanmaz, kaynaklar tükenmez.
- **Half-Open:** Belli süre sonra deneme. Başarılıysa Closed'a, başarısızsa tekrar Open'a.

Kütüphaneler: **Resilience4j** (modern), Hystrix (deprecated).

```
@CircuitBreaker(name = "paymentService", fallbackMethod = "fallback")
public PaymentResult charge(chargeRequest r) {
    return paymentClient.charge(r);
}

public PaymentResult fallback(chargeRequest r, Exception e) {
    return PaymentResult.queued(); // Asenkron kuyrukta dene
}
```

S: Retry ile Circuit Breaker birlikte nasıl çalışır?

C: Önce **Retry** (örn. 3 kez tekrar). Tüm denemeler başarısızsa Circuit Breaker'a sinyal verilir. Eşik aşılsa CB Open olur, sonraki istekler hiç denenmez. Bu kombinasyon en yaygın resilience setup'ıdır.

Saga Pattern (Distributed Transaction)

S: Distributed transaction'ı niye klasik @Transactional ile çözemeyiz?

C: Monolith'te tek DB var, ACID kolayca sağlanır. Microservice'lerde her servisin **kendi DB'si** var. Servis A commit etti, servis B çöktü → A'nın işlemini geri almak için DB rollback yetmez (çoktan commit olmuş). 2PC (Two-Phase Commit) bir çözüm ama lock contention nedeniyle ölçeklenmez. Modern yaklaşım: **Saga Pattern**.

S: Saga Pattern nedir?

C: Distributed transaction'ı bir **dizi local transaction**'a böler. Her local tx'in bir **compensating transaction**'ı vardır — başarısızlık durumunda önceki adımları **mantıksal olarak geri alır**. Yani gerçek rollback değil, **tersi işlem yapma**. Örnek: Ödeme alındı → ürün rezerve edildi → kargo başarısız → ürün rezervasyonu iptal + ödeme iadesi.

S: Choreography ve Orchestration tarzları?

C: Choreography: Merkezi koordinatör yok. Her servis event publish eder, ilgili servisler dinler. Decentralized, gevşek bağlı. Küçük-orta projeler için iyi. Dezavantaj: akış kodun her yerinde, debug zor, event boğulması olabilir.

Orchestration: Merkezi bir **orchestrator** akışı yönetir. Her servise sırayla komut gönderir, sonucu beklere, hata olursa compensating adımları tetikler. Karmaşık akışlar için temiz; büyük projelerde tercih edilir. Araçlar: Camunda, Temporal, Axon Framework.

```
// Choreography (Kafka örneği)
@Service
public class OrderService {
    public void createOrder(Order o) {
        orderRepo.save(o);
        kafkaTemplate.send("order-created", new OrderCreatedEvent(o.getId()));
    }
}
```



```
@Service
public class PaymentService {
    @KafkaListener(topics = "order-created")
    public void handle(OrderCreatedEvent e) {
        try { processPayment(e); }
        catch (Exception ex) {
            kafkaTemplate.send("payment-failed", new PaymentFailedEvent(e.getOrderId()));
        }
    }
}
// OrderService 'payment-failed' dinler ve order'ı CANCELLED yapar (compensating)
```

S: Saga'nın trade-off'ları?

C: Avantaj: Ölçeklenebilir, gerçek tx lock'u yok, eventual consistency.

Dezavantaj: Isolation yok — yarım adımdayken başka tx'ler veriyi görür. Compensating logic yazmak karmaşık. Idempotency önemli (aynı event 2 kez gelebilir, duplicate effect olmamalı).

Diğer Önemli Pattern'ler (Üst Düzey Bilgi)

Outbox Pattern: DB save + event publish'i atomic yapmak için. Event'i ayrı bir outbox tablosuna yaz (DB tx içinde), background job event'i kafka'ya push eder.

CQRS (Command Query Responsibility Segregation): Yazma ve okuma için farklı modeller. Karmaşık raporlama gereken sistemlerde.

Event Sourcing: Veriyi state olarak değil, olaylar dizisi olarak sakla. State, olayları replay ederek elde edilir.

BFF (Backend For Frontend): Her client tipi (mobile, web) için ayrı backend; client'a özel response birleştirme.

Distributed Tracing: Bir request'i tüm servislerde takip etmek için. Spring Cloud Sleuth + Zipkin/Jaeger ile.

14. Caching

S: Caching ne işe yarar?

C: Pahalı operasyonların (DB sorgusu, external API çağırısı, ağır hesaplama) sonucunu geçici saklayıp tekrarlı çağrılarda anında dönmek. **Response time** düşer, **throughput** artar, DB yükü azalır.

S: Spring'de caching nasıl yapılır?

C: @EnableCaching + @Cacheable / @CacheEvict / @CachePut anotasyonları.

- **@Cacheable:** Method'un dönüş değerini cache'ler. Aynı parametre ile çağrılırsa method ÇALIŞMAZ, cache'ten döner.
- **@CacheEvict:** Cache'ten kayıt **siler** (update/delete sonrası stale veriyi temizler).
- **@CachePut:** Method'u her zaman çalıştırır ama sonucu cache'e yazar (update senaryosu).

```
@Service
public class PersonelService {
    @Cacheable(value = "personel", key = "#id")
    public Personel getById(Long id) { ... }

    @CacheEvict(value = "personel", key = "#p.id")
    public void update(Personel p) { ... }

    @CacheEvict(value = "personel", allEntries = true)
    public void deleteAll() { ... }
}
```

Local vs Distributed Cache

S: Local cache ile distributed cache arasındaki fark?

Konu	Local (Caffeine, EhCache)	Distributed (Redis, Hazelcast)
Konum	JVM içinde	Ayrı server (network)
Hız	Nanosaniye	Milisaniye (network latency)
Paylaşım	Yalnız o instance	Tüm instance'lar paylaşır
Restart	Veri kaybolur	Veri kalır (persistence opsiyonu)
Multi-instance tutarlılığı	Problem (her instance farklı veri)	Tutarlı
TTL	Var	Var

S: Hangisini ne zaman seçersin?

C: Local: Tek instance, read-heavy, küçük data, kısa TTL, max performans gerek.

Distributed: Multi-instance/scaled deployment, session sharing, büyük data, deployment'a dayanıklı olması gereken cache, instance'lar arası tutarlılık.

Redis Özelinde

S: Redis nedir?

C: In-memory key-value store. **Tek thread** ana event loop'la çalışır ama **çok hızlıdır** (microsaniye seviyesinde). Data type'ları: String, List, Set, Sorted Set, Hash, Stream. Cache dışında **pub/sub**, **distributed lock**, **rate limiting** için de kullanılır.

S: Cache stratejileri nelerdir?

C: Cache-Aside (Lazy Loading): Uygulama önce cache'e bakar; yoksa DB'den okur, cache'e yazar, döner. En yaygın.

Read-Through: Cache layer'ı DB'yi okumayı kendi yapar, uygulama sadece cache ile konuşur.

Write-Through: Yazma anında hem cache hem DB'ye yazılır. Tutarlılık yüksek.

Write-Behind (Write-Back): Önce cache'e yazılır, asenkron olarak DB'ye flush edilir. Performans yüksek, ama cache fail olursa veri kaybı riski.

S: Cache invalidation problemleri?

C: Phil Karlton'un meşhur sözü: "Computer science'ta iki zor şey vardır: cache invalidation ve isimlendirme."

Yaygın stratejiler:

- **TTL (Time-To-Live):** X saniye sonra otomatik expire.
- **Manual invalidation:** Update yapılıncaya cache'ten sil (@CacheEvict).
- **Event-based:** Domain event yayınla, cache servisi dinleyip sil.

15. Test (Kısa)

Senior pozisyonlarda test yazabilmek beklenir. Burada özet düzeyde anlatıyorum, mülakatta sorulursa temel kavramları söyleyebilmen yeterli.

S: Unit Test, Integration Test, E2E Test farkı?

C: Unit Test: Tek bir sınıf/method'u izole test eder. Bağımlılıklar mock'lanır. Hızlı, çok.

Integration Test: Birden çok bileşenin birlikte çalışmasını test eder (örn. Service + Repository + gerçek DB). Yavaş ama daha gerçekçi.

E2E Test: Kullanıcı senaryosunu baştan sona test eder. Çok yavaş, kırılgan ama iş değeri yüksek.

Test Pyramid: Çok unit, az integration, çok az E2E.

S: JUnit 5 ile Mockito örneği?

```
@ExtendWith(MockitoExtension.class)
class IzinServiceTest {
    @Mock IzinRepository repo;
    @Mock NotificationSender notif;
    @InjectMocks IzinService service;

    @Test
    void izinOnayla_basariliSenaryo() {
        // given
        Izin izin = new Izin(1L, "BEKLEMEDE");
        when(repo.findById(1L)).thenReturn(Optional.of(izin));

        // when
        service.onayla(1L);

        // then
        assertEquals("ONAYLANDI", izin.getDurum());
        verify(notif).gonder(any());
    }
}
```

S: @Mock, @InjectMocks, @Spy farkı?

C: @Mock: Tamamen sahte nesne. Metotlar default değer döner; when()...thenReturn() ile davranış tanımlanır.

@InjectMocks: Gerçek nesne yaratır, içine @Mock'ları enjekte eder. Test ettiğin asıl sınıf.

@Spy: Gerçek nesneyi sarmalar. Default real metot çalışır, istediğin metodu stub'layabilirsin (partial mocking).

S: Spring Boot test araçları?

C: @SpringBootTest: Tam application context yükler — integration test için.

@WebMvcTest: Sadece web layer'ı yükler, MockMvc ile controller test.

@DataJpaTest: Sadece JPA layer'ı yükler, in-memory H2 ile repository test.

Testcontainers: Docker container'da gerçek Postgres/Redis/Kafka çalıştırır, modern integration test standardı.

S: TDD (Test-Driven Development) nedir?

C: Önce test yaz (fail), sonra geçecek minimum kodu yaz (pass), sonra refactor et. **Red-Green-Refactor** döngüsü. Avantajları: kapsama otomatik, tasarımı kullanılabilir kılar, regresyon güvencesi.

16. Clean Code & Refactoring

S: Clean Code prensipleri nelerdir?

C: Robert C. Martin'in "Clean Code" kitabından özetle:

- **Anlamlı isimler:** `d` değil `elapsedDays`. Yorum yazmana gerek bırakmayan isimler.
- **Küçük metotlar:** Bir metot 20 satırı geçmesin, tek iş yapsın (SRP).
- **Az parametre:** 0-2 ideal, 3'ten fazla parametre kötü kokar — DTO/parameter object kullan.
- **Yorum yerine kod konuşsun:** İyi isimler yorumun çoğunu gereksiz kılar.
- **DRY (Don't Repeat Yourself):** Aynı mantığı iki yerde yazma.
- **KISS (Keep It Simple, Stupid):** En sade çözüm en iyisidir.
- **YAGNI (You Aren't Gonna Need It):** Şu anda gerekmeyeni yazma.

S: Code smell nedir? Örnek ver.

C: Kodun "yanlış" olduğunu sezdirenen işaretler. Klasikleri:

- **Long Method:** 100 satırlık metot.
- **God Class:** Her şeyi yapan dev sınıf (SRP ihlali).
- **Long Parameter List:** 7-8 parametre.
- **Duplicate Code:** Aynı blok kopyala-yapıştır.
- **Switch Statement Cehennemi:** Aynı switch farklı yerlerde — polymorphism eksikliği.
- **Feature Envy:** Bir sınıf başka sınıfın field'larını sürekli kullanıyor → metodu o sınıfa taşı.
- **Primitive Obsession:** String/int kullanıp her yere validation koymak yerine value object yarat (örn. Email, Money sınıfları).
- **Magic Numbers:** `if (status == 3)` yerine `if (status == OrderStatus.SHIPPED)`.

S: Coupling ve Cohesion nedir?

C: Coupling (Bağıllık): Sınıfların birbirine ne kadar bağlı olduğu. **Düşük olmalı.** Yüksek coupling → bir sınıfı değiştiren başkaları kırılır.

Cohesion (Uyum): Bir sınıfın elemanlarının ne kadar ilgili olduğu. **Yüksek olmalı.** Düşük cohesion → sınıf birbiriyle alakasız işler yapıyor (SRP ihlali).

Hedef: *Loose coupling, high cohesion.*

S: Refactoring ne demek? Risk almak gerekir mi?

C: Davranış değişmeden kodun yapısını iyileştirme. Test coverage olmadan refactoring riskli; önce test ekle, sonra refactor et. Tipik teknikler: Extract Method, Rename, Move Method, Replace Conditional with Polymorphism, Introduce Parameter Object.

S: Bir method ne kadar uzun olmalı, kaç parametre alabilir?

C: İdeal: Bir ekrana sığacak kadar, 10-20 satır. Geçtiyse extract method yap. **Parametre:** 0-2 ideal, 3 kabul edilebilir, 4+ kötü koku. Çok parametre varsa DTO/builder/Parameter Object kullan.

S: Hot spot olan büyük bir method'u nasıl refactor edersin?

1. **Önce test yaz** — mevcut davranışı korumak için güvenlik ağı.
2. **Anlamlı bloklara böl** — yorum yazdığın yerine Extract Method.
3. **Anlamlı isimler ver.**

-
4. **Conditional logic'i polymorphism'a çevir** — büyük switch'i Strategy'e dönüştür.
 5. **SRP'ye böl** — sınıf tek iş yapsın.
 6. **Adım adım commit et**, her commit'te test yeşil kalsın.

□ **İpucu:** *Senin geçmiş debugging deneyimlerinden örnek verebilirsin: "Tosyalı half-day leave bug'ında WfizinCommand'da iki overlapping conflict check vardı, ortak guard pattern ile extract method yapıp tek noktadan yöneterek refactor ettim."*

17. Behavioral / Soft Skill Soruları

Akbank gibi büyük kurumlarda teknik mülakat sonrası bir İK / yönetici mülakatı olur. Aşağıdaki sorulara önceden cevap hazırlamak çok kıymetli.

Klasik İK Soruları

S: En zorlandığınız bir projeyi/anı anlatın.

C: STAR formatıyla: Situation, Task, Action, Result.

"ARGUS izin yönetiminde onay tarihlerinin 00:00:00 kaydedildiği bir bug vardı. (S)

Görevim hem mevcut çağrılan onlarca yeri kırmadan hem de zamanı doğru kaydederek düzeltmekti. (T)

Sonuç olarak `java.sql.Timestamp`'ın `Date`'i extend ettiğini keşfedip, metot imzasını bozmadan `Timestamp` instance döndürerek çözdüm. (A)

Bu sayede setter'ları değiştirmeden tüm tarih+zaman doğru kaydedildi, hiçbir yer kırılmadı. (R)"

S: Bir takım arkadaşıyla anlaşmazlık yaşadığınız bir an oldu mu? Nasıl çözdünüz?

C: Mümkünse gerçek bir örnek ver. Anlaşmazlığı **teknik bir konuya** bağla (mimari tercih, kütüphane seçimi gibi), kişisel kavga gibi gösterme. "Karşı tarafı dinledim, ortak noktayı bulduk, kanıt + örnek üzerinden konuştuk, gerekirse PoC yaptık" şeklinde olgun bir yaklaşım sergile.

S: Bir hatadan ne öğrendiniz?

C: Production'a giden bir bug, eksik test, yanlış varsayım gibi gerçek bir örnek. Tepki: "Suçluyu aramak yerine root cause analizi yaptım, postmortem yazdım, regression test ekledim, takıma deneyimi paylaştım."

S: Beş yıl sonra kendinizi nerede görüyorsunuz?

C: Senior/tech lead seviyesinde, takım mentorluğu yapan, mimari kararlara katkı sunan, iş hedeflerini teknolojiyle birleştirebilen biri. Karşımdaki şirketin büyüme yolunda kendimi konumlamış olmak.

S: Maaş beklentiniz nedir?

C: Önceden piyasa araştırması yap (kariyer.net, glassdoor, türkçe Reddit r/cscareerquestions). Bir aralık ver: "Şu seviyede deneyimle X-Y TL bandında bekliyorum, fakat toplam paket (prim, yan haklar, kariyer fırsatı) bütünüyle değerlendiriyorum." İlk rakamı söyletme — "Şirketinizin bu pozisyon için bandı nedir?" diye sorabilirsin.

S: Neden iş değiştirmek istiyorsunuz?

C: Negatif söyleme tuzağına düşme! Mevcut firmaya küfretmek kötü izlenim bırakır. Pozitif söyle: "Yeni teknolojilerle çalışma fırsatı, daha büyük ölçekli sistemler tecrübesi, kariyer büyümesi, yeni iş alanları öğrenme."

S: Güçlü ve zayıf yanlarınız?

C: Güçlü: Sorun çözme, detaycı yaklaşım, öğrenmeye açıklık, takım çalışması (somut örneklerle destekle).

Zayıf: Klişe "mükemmeliyetçilik" deme. Gerçek bir zayıflık + onu nasıl geliştirdiğini söyle: "Public speaking'de zorlanırdım, tech talk'lara katılarak ve sprint demolar vererek kendimi geliştirdim."

Akbank'a Özel Sorulabilecekler

S: Bankacılık sektöründe çalışmaya yatkınlığınız var mı?

C: Vurgula: regülasyon hassasiyeti (KVKK, BDDK), veri tutarlılığı önemi (ACID), yüksek güvenlik gereksinimi, finansal okuryazarlık (öğrenmeye açık olduğunu söyle), scale (milyonlarca müşteri) ilgini çekiyor.

S: Akbank hakkında ne biliyorsunuz?

C: Mülakat öncesi araştır: kuruluşu, holding (Sabancı), dijital bankacılıkta öncülüğü (Akbank Direkt), AI/data yatırımları, son projeleri (haber/LinkedIn'den), tech yığını (LinkedIn'deki personel profillerinden çıkarsanır).

18. Sen Ne Sorabilirsin?

"Sorunuz var mı?" mülakatın sonunda mutlaka gelir. "Yok" demek **HATA**. Hazır sorularla geldiğini göstermek ciddiyetini ispatlar. İşte güçlü sorular:

□ Teknik Sorular (Tech Lead/Manager'a)

1. **Takımın şu anki tech stack'i nedir?** Java sürümü, Spring sürümü, hangi DB, hangi messaging?
2. **Monolith mi microservice mi?** Geçiş süreciniz var mı?
3. **Test kültürünüz nasıl?** Code coverage hedefi var mı? TDD uyguluyor musunuz?
4. **Code review süreci nasıl işliyor?** Kaç onay gerekiyor, otomasyon var mı?
5. **CI/CD pipeline'iniz nasıl?** Deployment sıklığı ne?
6. **On-call rotasyonu var mı?** Production incident'a nasıl müdahale ediliyor?
7. **Observability stack'iniz?** Prometheus/Grafana, ELK, Jaeger?
8. **Yeni başlayan biri için onboarding süreci nasıl?**

□ Takım & Kültür Soruları

1. **Takımın büyüklüğü ve yapısı nedir?** Senior/junior dağılımı?
2. **Agile metodolojiniz nedir?** Scrum mu, Kanban mı? Sprint süresi?
3. **Bir tipik haftam nasıl geçer?** Toplantı yoğunluğu, focus time?
4. **Hibrit/uzaktan çalışma modeliniz nedir?**
5. **Takımın en güçlü ve geliştirilebilecek yanları sizce neler?**

□ Kariyer Gelişimi Soruları

1. **Bu pozisyondan 1-2 yıl içinde nereye gidilebilir?** Promotion yolu nasıl?
2. **Mentorluk/eğitim bütçesi var mı?** Konferans, kurs desteği?
3. **Performans değerlendirmesi nasıl yapılıyor?** Hangi metrikler?
4. **İç eğitimler veya tech talk'lar var mı?**

□ Bu Rolün Özünü Anlama Soruları

1. **Bu pozisyondaki ilk 30/60/90 günde benden ne bekliyorsunuz?**
2. **Bu rolde başarının ölçüsü nedir?**
3. **Bu rolde en zorlu kısım sizce nedir?**
4. **Önceki kişi neden ayrıldı / pozisyon neden açıldı?**

□ Süreç Sorusu (En Sonda)

1. **Sonraki adımlar nelerdir?** Ne zaman dönüş alabilirim?
2. **Sizinle başka bir görüşme olacak mı?**

□ **İpucu: Tavsiye:** En az 3-5 soru hazırlamış ol. Hepsini sormaya gerek yok — mülakatçı zaten birkaçını cevaplamış olabilir, sen kalanlardan seçersin.
Hazırlandığını gösteren, ilgili ve düşünülmüş sorular sormak **en önemli bonus**'tur.

BAŞARILAR BATUR! ☐

"İyi bir mülakat, iyi bir hazırlığın eseridir."

Bu dökümanı 2-3 kez baştan sona okuyup üstüne küçük notlar düşersen, mülakatta her zorlu soruda kafanda bir çıkış noktan olur. Bilmediğin sorularda dürüstçe "şu an net bilmiyorum ama şu yönde tahmin yürütüyorum" demek, ezberden yanlış cevap vermekten her zaman iyidir.